

Apostila

Lógica Reconfigurável com VHDL

Fundamentos e Aplicações

Dispositivos lógicos programáveis, descrição de hardware e simulação

Prof. Felipe Walter Dafico Pfrimer

Objetivo

Esta apostila apresenta os fundamentos de lógica reconfigurável com VHDL, introduzindo PLDs, HDLs, estrutura básica da linguagem, simulação e o fluxo inicial de projeto digital.

26 de maio de 2026

Sumário

1	Introdução	1
1.1	O que são Dispositivos Lógicos Programáveis?	1
1.2	Linguagens de Descrição de Hardware	2
1.3	O que é VHDL?	2
1.4	Fluxo básico de projeto com VHDL	5
1.4.1	Ferramentas utilizadas na disciplina	5
1.4.2	Próximos capítulos	6
1.5	Perguntas para revisão	6
1.6	Leitura complementar: histórico e evolução da linguagem VHDL	6
1.6.1	Origens e o Programa VHSIC (Década de 1980)	6
1.6.2	Padronização e Evolução (IEEE)	7
1.6.3	Estado da Arte e Aplicação Atual	7
2	Ferramentas e ambiente de desenvolvimento	9
2.1	Visão geral do ambiente	9
2.2	Organização dos arquivos de projeto	10
2.3	Instalação do Visual Studio Code	11
2.4	Configuração da extensão VHDL for Professionals	13
2.5	Instalação do GHDL	14
2.6	Primeira simulação com GHDL	15
2.7	Instalação do VaporView	15
2.8	Visualização de formas de onda	15
2.9	O papel do Quartus no fluxo completo	15
2.10	Automação futura com Tcl	16
2.11	Fluxo de trabalho recomendado	16
2.12	Problemas comuns e verificação da instalação	16
2.13	Perguntas para revisão	16

Capítulo 1

Introdução

Esta apostila apresenta conceitos fundamentais de VHDL utilizados na disciplina. Este capítulo introduz a base tecnológica dos Dispositivos Lógicos Programáveis (PLDs), a importância das Linguagens de Descrição de Hardware (HDLs) e os primeiros elementos da linguagem VHDL.

1.1 O que são Dispositivos Lógicos Programáveis?

Imagine uma bancada formada por vários módulos fixos: pequenas portas lógicas, registradores, multiplexadores e fios que podem ser conectados por meio de chaves configuráveis. A bancada física é sempre a mesma; os módulos não mudam de lugar nem se transformam em outros componentes. O que muda, a cada projeto carregado, é quais conexões ficam ativas entre esses módulos.

Com uma configuração, os módulos podem ser conectados para formar um contador. Com outra configuração, os mesmos módulos podem formar um somador, um controlador digital ou parte de um sistema mais complexo. Portanto, a flexibilidade não vem de trocar fisicamente o hardware, mas de reconfigurar as conexões internas entre recursos já existentes.

Essa ideia ilustra o funcionamento dos **Dispositivos Lógicos Programáveis** (PLDs – *Programmable Logic Devices*): componentes eletrônicos formados por blocos lógicos e interconexões programáveis. Ao carregar um novo “projeto” — ou seja, um novo arquivo de configuração — o usuário define como esses blocos serão conectados para executar uma função lógica específica.

No contexto dos PLDs, os blocos lógicos correspondem aos recursos básicos disponíveis no dispositivo, enquanto as interconexões programáveis funcionam como caminhos configuráveis entre eles. O projeto digital define quais conexões devem ser estabelecidas e como os blocos serão usados. Esse projeto pode ser descrito tanto por diagramas esquemáticos quanto por **Linguagens de Descrição de Hardware** (HDLs).

Ideia-chave

Em um PLD, o hardware não executa uma sequência de instruções como um processador comum. Ele é configurado para se comportar como um circuito digital específico.

Existem diversas categorias de PLDs. Entre as mais comuns estão os **FPGAs** (*Field-Programmable Gate Arrays*) e os **CPLDs** (*Complex Programmable Logic Devices*). Os FPGAs são conhecidos por sua alta densidade lógica e flexibilidade, permitindo a implementação de sistemas digitais complexos. Já os CPLDs costumam ser mais simples, possuem arquitetura menos granular e são ideais para lógicas de controle e aplicações que exigem previsibilidade temporal rigorosa.

1.2 Linguagens de Descrição de Hardware

Uma HDL (*Hardware Description Language*) **não é uma linguagem de programação**. Como o próprio nome sugere, trata-se de uma linguagem utilizada para **descrever** o hardware. Diferentemente das linguagens de software (como C ou Python), que geram instruções sequenciais para um processador, as HDLs permitem que engenheiros definam a estrutura e o comportamento de circuitos eletrônicos, onde muitas coisas acontecem simultaneamente (concorrência).

Antes de continuar

Ao ler exemplos em VHDL, evite procurar uma ordem de execução linha por linha, como em C ou Python. O ponto principal é identificar quais sinais existem, como eles se conectam e que comportamento de hardware está sendo descrito.

Atualmente, as HDLs mais populares para descrever circuitos digitais em PLDs são o **VHDL** (*VHSIC Hardware Description Language*), o **Verilog** e o **SystemVerilog**. Todas permitem a descrição de circuitos em diferentes níveis de abstração, desde o comportamental (o que o circuito faz) até o estrutural (quais portas lógicas compõem o circuito). Esta apostila foca no uso do **VHDL**.

1.3 O que é VHDL?

A sigla VHDL significa **VHSIC Hardware Description Language**, onde VHSIC é o acrônimo para *Very High Speed Integrated Circuit*. Desenvolvida na década de 1980 pelo Departamento de Defesa dos Estados Unidos, a linguagem foi criada para documentar e simular o comportamento de circuitos integrados complexos. Com o tempo, o VHDL evoluiu e se tornou um padrão internacional (IEEE 1076), amplamente adotado na indústria eletrônica para o design de sistemas digitais.

Para exemplificar, considere o Código 1.1, escrito em VHDL, que descreve uma porta AND de duas entradas. Leia este primeiro exemplo como um mapa: os detalhes serão explicados logo abaixo, mas já é possível observar a divisão entre a interface do circuito e seu comportamento interno.

Antes da leitura do código

Antes de tentar entender cada palavra, procure três regiões: as bibliotecas usadas, a entidade que declara entradas e saídas, e a arquitetura que descreve o comportamento da saída.

Código 1.1: *Descrição VHDL de uma porta AND de duas entradas*

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity AND2 is
5     port (
6         A, B : in  std_logic;
7         Y    : out std_logic
8     );
9 end AND2;
```

```
10
11 architecture rtl of AND2 is
12 begin
13     Y <= A and B;
14 end rtl;
```

Depois da leitura do código

A entidade diz como o circuito aparece para o mundo externo. A arquitetura diz o que acontece dentro dele. Essa separação aparece em praticamente todos os projetos VHDL.

Observe no Código 1.1 a estrutura básica de um arquivo VHDL:

- **Bibliotecas:** As linhas iniciais importam a biblioteca `ieee`, necessária para usar o tipo de dado padrão `std_logic`. Outras bibliotecas podem ser incluídas conforme a complexidade do projeto.
- **Entidade (Entity):** Define a interface do componente, ou seja, suas entradas (A, B) e saídas (Y). Funciona como a “caixa preta” do componente vista de fora. Aqui, o projetista deve prever quais sinais o componente receberá e quais ele fornecerá.
- **Arquitetura (Architecture):** Descreve o comportamento interno. Neste caso, a saída Y recebe o resultado da operação lógica AND entre A e B. É nessa região que os iniciantes em VHDL possuem maior dificuldade, pois já estão acostumados a pensar em termos de sequências de comandos comuns nas linguagens de programação. Dessa forma, o VHDL exige uma mentalidade de descrição concorrente. Na arquitetura, todas as operações são consideradas simultâneas, refletindo a natureza paralela do *hardware* alvo.

Além da descrição para síntese (criação do circuito físico), o VHDL é amplamente utilizado para **simulação**. Isso permite que os desenvolvedores validem o comportamento do circuito no computador antes de implementá-lo no PLD, economizando tempo e garantindo a confiabilidade do projeto.

O código a seguir apresenta um exemplo simples de testbench em VHDL, utilizado para simular o comportamento da porta AND definida no Código 1.1. Ele contém mais elementos que o exemplo anterior; neste momento, o mais importante é perceber que o testbench cria sinais, conecta a porta AND e aplica combinações de entrada ao longo do tempo.

Ao ler o testbench

Não é necessário memorizar a estrutura completa agora. Observe apenas que os comandos `wait for 10 ns` criam intervalos de simulação e permitem verificar como a saída responde a cada combinação de A e B.

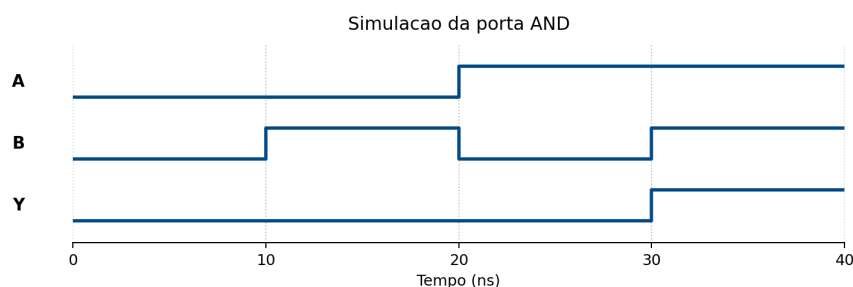
Código 1.2: Testbench para validar a porta AND

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity tb_AND2 is
```

```
6 end tb_AND2;
7
8 architecture behavior of tb_AND2 is
9     signal A, B : std_logic := '0';
10    signal Y      : std_logic;
11
12    component AND2
13        port (
14            A, B : in  std_logic;
15            Y    : out std_logic
16        );
17    end component;
18 begin
19    uut: AND2 port map (A => A, B => B, Y => Y);
20
21    process
22    begin
23        A <= '0'; B <= '0'; wait for 10 ns;
24        A <= '0'; B <= '1'; wait for 10 ns;
25        A <= '1'; B <= '0'; wait for 10 ns;
26        A <= '1'; B <= '1'; wait for 10 ns;
27        wait;
28    end process;
29 end behavior;
```

No Código 1.2, o testbench cria sinais de entrada (A e B) e conecta a unidade sob teste (UUT - *Unit Under Test*), que é a porta AND definida anteriormente. O processo dentro do testbench aplica diferentes combinações de entradas, aguardando 10 nanosegundos entre cada mudança, permitindo observar a saída Y em resposta às entradas. O resultado da simulação pode ser visualizado em uma forma de onda, facilitando a verificação do comportamento correto do circuito, como pode ser visto na Figura 1.1.

Figura 1.1: *Forma de onda da simulação da porta AND*



Dessa forma, o VHDL possui duas formas principais de uso:

- **Síntese:** na qual o código VHDL infere o circuito lógico que será implementado no PLD.
- **Simulação:** na qual o código VHDL é utilizado para validar o comportamento do circuito antes da síntese.

Essas duas vertentes são fundamentais no fluxo de projeto digital, garantindo que o design atenda aos requisitos funcionais e de desempenho antes da implementação física. No entanto, cada uma dessas vertentes possui suas próprias regras e boas práticas de codificação, que serão abordadas nos capítulos seguintes. Isso acontece porque certos construtos do VHDL são adequados para simulação, mas não para síntese, e vice-versa. Por exemplo, estruturas como `wait` e `after` são úteis para simulação, mas não podem ser sintetizadas em hardware.

1.4 Fluxo básico de projeto com VHDL

Em um projeto digital com VHDL, o código escrito pelo projetista passa por uma sequência de etapas antes de chegar ao hardware. Em uma visão simplificada, esse fluxo pode ser entendido da seguinte forma:

Descrição em VHDL → Simulação → Síntese → Implementação no PLD → Teste em hardware

Na etapa de **descrição**, o projetista escreve os arquivos VHDL que representam o circuito desejado. Em seguida, na **simulação**, um testbench é usado para verificar se o comportamento descrito está correto antes de envolver o hardware físico. Se a simulação indicar que o circuito se comporta como esperado, o projeto pode seguir para a **síntese**, etapa em que a descrição VHDL é convertida em uma rede de elementos lógicos.

Depois da síntese, a ferramenta de desenvolvimento realiza a **implementação**, posicionando e conectando esses elementos dentro do PLD escolhido. Por fim, o projeto é carregado no dispositivo e testado em hardware real. Na prática, esse fluxo costuma ser iterativo: erros encontrados na simulação, na síntese ou no teste em hardware levam o projetista de volta ao código VHDL.

Ideia-chave

Simular antes de sintetizar reduz o tempo gasto procurando erros no hardware. Sempre que possível, o comportamento do circuito deve ser validado no computador antes da implementação física.

1.4.1 Ferramentas utilizadas na disciplina

Ao longo da disciplina, serão utilizadas ferramentas para edição, simulação, visualização de formas de onda e implementação em FPGA. A organização prevista é:

- **Visual Studio Code:** ambiente de edição dos arquivos VHDL.
- **GHDL:** ferramenta para compilação e simulação dos projetos VHDL.
- **VaporView:** ferramenta para visualização de formas de onda geradas nas simulações.
- **VHDL for Professionals:** extensão de apoio à edição e análise de código VHDL.
- **Quartus:** ferramenta para síntese, implementação e gravação dos projetos em FPGA.
- **Tcl:** possível apoio para automatizar comandos e fluxos dentro do Quartus.

1.4.2 Próximos capítulos

Os próximos capítulos irão aprofundar gradualmente os elementos apresentados nesta introdução. Esta seção será completada conforme a estrutura final da apostila for definida.

1.5 Perguntas para revisão

1. O que significa dizer que um PLD pode ser configurado pelo usuário?
2. Na analogia da bancada modular, o que permanece fixo e o que muda quando um novo projeto é carregado?
3. Por que uma HDL não deve ser entendida como uma linguagem de programação comum?
4. Quais HDLs são citadas como populares para descrever circuitos digitais em PLDs?
5. Qual é a função da entidade em um código VHDL?
6. Qual é a função da arquitetura em um código VHDL?
7. No exemplo da porta AND, quais são as entradas e qual é a saída?
8. Para que serve um testbench?
9. Qual é a diferença entre síntese e simulação?
10. Por que estruturas como `wait` e `after` são úteis para simulação, mas não para síntese?

1.6 Leitura complementar: histórico e evolução da linguagem VHDL

Esta seção apresenta um breve contexto histórico sobre a origem do VHDL. Ela não é necessária para compreender os exemplos anteriores, mas ajuda a entender por que a linguagem possui características como tipagem forte, sintaxe rigorosa e foco em documentação de hardware.

A linguagem VHDL (*VHSIC Hardware Description Language*) possui uma origem distinta da maioria das linguagens de programação de software, tendo sido concebida inicialmente não para a execução, mas para a documentação e preservação de projetos eletrônicos complexos [1].

1.6.1 Origens e o Programa VHSIC (Década de 1980)

No início da década de 1980, o Departamento de Defesa dos Estados Unidos (DoD) enfrentava uma crise logística relacionada ao ciclo de vida de seus componentes eletrônicos. Com o avanço rápido da tecnologia, chips utilizados em equipamentos militares tornavam-se obsoletos rapidamente, e a falta de documentação padronizada impedia a reprodução desses componentes por novos fabricantes [7, 3].

Para solucionar este problema, o DoD iniciou o programa VHSIC (*Very High Speed Integrated*

Circuits). O objetivo era criar uma linguagem agnóstica de tecnologia que pudesse descrever o comportamento e a estrutura de circuitos integrados. Em 1983, um contrato foi firmado com a Intermetrics, IBM e Texas Instruments para desenvolver a primeira versão da linguagem. Uma decisão crucial de design foi basear a sintaxe na linguagem Ada, herdando desta a tipagem forte e a não-sensibilidade a letras maiúsculas/minúsculas, visando reduzir erros de ambiguidade em projetos críticos [6].

1.6.2 Padronização e Evolução (IEEE)

Embora criada para documentação, os engenheiros perceberam rapidamente que uma descrição comportamental precisa poderia ser executada por softwares de simulação. Para fomentar a adoção pela indústria civil e garantir a interoperabilidade entre ferramentas de CAD (*Computer-Aided Design*), o DoD transferiu os direitos da linguagem para o IEEE (*Institute of Electrical and Electronics Engineers*) em 1986.

O primeiro padrão oficial foi publicado como **IEEE Standard 1076-1987** (conhecido como VHDL-87). Desde então, a linguagem passou por revisões periódicas para incorporar novas capacidades de síntese e verificação [5]:

- **VHDL-93:** Introduziu uma sintaxe mais consistente, identificadores estendidos e melhorias na manipulação de arquivos.
- **IEEE 1164:** Embora não seja uma revisão da linguagem em si, a padronização do pacote `std_logic_1164` foi um marco complementar fundamental. Ela introduziu o sistema lógico de 9 valores (incluindo 'Z' para alta impedância e 'X' para desconhecido), permitindo a modelagem realista de circuitos digitais.
- **VHDL-2000 e 2002:** Pequenas revisões que adicionaram o modificador `protected` para tipos.
- **VHDL-2008 (IEEE 1076-2008):** Uma grande modernização que incorporou recursos inspirados em SystemVerilog, facilitando a verificação e reduzindo a verbosidade do código para operações comuns.
- **VHDL-2019:** Uma revisão recente, focada em melhorias para interfaces de verificação e introspecção de tipos.

1.6.3 Estado da Arte e Aplicação Atual

Atualmente, o VHDL é uma das linguagens dominantes no design de hardware digital, dividindo o mercado global com o Verilog/SystemVerilog. Seu uso principal migrou da documentação para a **síntese lógica** e a **simulação**.

O VHDL é amplamente empregado no desenvolvimento de FPGAs (*Field-Programmable Gate Arrays*) e ASICs (*Application-Specific Integrated Circuits*). Devido à sua natureza fortemente tipada e determinística, o VHDL mantém uma forte preferência em setores que exigem alta confiabilidade e segurança crítica, como as indústrias aeroespacial, automotiva, médica e de defesa, especialmente na Europa e nas Américas. Ferramentas modernas de síntese da Intel (Quartus) e AMD/Xilinx (Vivado) oferecem suporte abrangente aos padrões mais recentes, permitindo que a linguagem descreva desde portas lógicas simples até processadores *soft-core* complexos e aceleradores de inteligência artificial [2, 4].

Capítulo 2

Ferramentas e ambiente de desenvolvimento

No capítulo anterior, o VHDL foi apresentado como uma linguagem usada para descrever circuitos digitais. A partir deste ponto, além de entender a linguagem, é necessário preparar um ambiente de trabalho capaz de editar os arquivos, simular o comportamento do circuito e visualizar formas de onda. A implementação em FPGA será tratada mais adiante, quando os projetos já tiverem passado por simulações simples.

Este capítulo apresenta as ferramentas utilizadas no início da disciplina e propõe uma organização inicial para os arquivos dos projetos. A ideia não é transformar a instalação das ferramentas no objetivo principal, mas criar um fluxo simples e repetível para que cada exemplo da apostila possa ser editado, testado e documentado.

Ideia-chave

O ambiente de desenvolvimento deve ajudar o aluno a verificar o circuito antes de pensar na placa. A simulação nem sempre é uma etapa obrigatória em todos os projetos, mas é altamente recomendável para detectar erros antes da síntese. Por isso, nesta apostila, ela fará parte do fluxo de trabalho adotado.

2.1 Visão geral do ambiente

O fluxo adotado no início desta apostila combina ferramentas de edição, simulação e visualização. Outras ferramentas, como o Quartus Prime Lite, aparecem aqui apenas para situar o fluxo completo que será usado posteriormente. Cada ferramenta cumpre um papel diferente:

- **Visual Studio Code:** editor utilizado para criar e organizar os arquivos VHDL.
- **VHDL for Professionals:** extensão do Visual Studio Code que auxilia na leitura, navegação e análise de arquivos VHDL.
- **GHDL:** ferramenta usada para analisar, elaborar e simular descrições VHDL.
- **VaporView:** visualizador de formas de onda geradas durante a simulação.
- **Quartus Prime Lite:** ferramenta da Intel que será usada mais adiante para sintetizar, implementar e gravar projetos em FPGAs compatíveis.
- **Tcl:** linguagem de scripts que poderá ser usada futuramente para automatizar comandos em ferramentas como o Quartus.

Essas ferramentas podem ser entendidas como partes de uma cadeia de trabalho. Primeiro,

o projetista escreve os arquivos VHDL no editor. Em seguida, usa o GHDL para simular o circuito e gerar resultados. Quando a simulação produz uma forma de onda, o VaporView permite observar se as entradas e saídas se comportaram como esperado. A implementação em FPGA entra apenas depois dessa base estar bem estabelecida.

Edição → Simulação → Forma de onda → Síntese futura → FPGA

Antes de instalar tudo

Nem toda ferramenta será usada com a mesma frequência no início da disciplina. Nos primeiros exemplos, o foco estará em editar arquivos VHDL, rodar simulações com GHDL e interpretar formas de onda. A instalação e o primeiro uso do Quartus serão tratados em um capítulo próprio, quando a apostila chegar à implementação em FPGA.

2.2 Organização dos arquivos de projeto

Antes de criar o primeiro projeto, é importante adotar uma estrutura de pastas simples. Isso evita confusão entre arquivos de código, arquivos de simulação, imagens, relatórios e saídas geradas automaticamente pelas ferramentas.

Uma organização recomendada para os exercícios da disciplina é:

Código 2.1: *Estrutura sugerida para um projeto VHDL simples.*

```
1 meu_projeto/  
2   src/  
3     porta_and.vhd  
4   sim/  
5     tb_porta_and.vhd  
6   waves/  
7     porta_and.vcd  
8   docs/  
9     anotacoes.md
```

Nessa estrutura, a pasta **src** guarda os arquivos VHDL que descrevem o circuito a ser sintetizado. A pasta **sim** guarda os *testbenches* (bancadas de teste) e outros arquivos usados apenas para simulação. A pasta **waves** guarda arquivos de forma de onda, como **.vcd**. A pasta **docs** pode ser usada para anotações, tabelas de pinos, observações de teste ou pequenos relatórios.

Para projetos maiores, outras pastas podem ser adicionadas, como **constraints** para arquivos de restrições e **quartus** para arquivos específicos da ferramenta de síntese. No início, porém, uma estrutura pequena é suficiente e ajuda a reforçar uma separação importante: nem todo arquivo usado durante a simulação faz parte do circuito que será sintetizado.

Síntese e simulação

Arquivos de *testbenches* normalmente não são sintetizáveis. Eles servem para estimular o circuito durante a simulação, mas não representam hardware que será gravado no FPGA. O próximo capítulo será dedicado a entender melhor o papel dos *testbenches* e como eles se encaixam no fluxo de desenvolvimento.

2.3 Instalação do Visual Studio Code

O Visual Studio Code, também chamado de VS Code, é um editor de código desenvolvido pela Microsoft. Ele não é um simulador de VHDL e também não substitui ferramentas de síntese, como o Quartus. Seu papel neste fluxo é servir como ambiente de edição: nele serão criados, organizados e modificados os arquivos `.vhd`, `.vhd1`, scripts e demais arquivos do projeto.

A Figura 2.1 mostra o ícone oficial do Visual Studio Code. Ele pode ajudar a identificar o aplicativo correto durante a instalação, principalmente porque o nome é parecido com o do Visual Studio.



Figura 2.1: Ícone oficial do Visual Studio Code.

Para instalar o Visual Studio Code:

1. Acesse a página oficial de download: <https://code.visualstudio.com/download>. A Figura 2.2 mostra essa página.
2. Na área correspondente ao seu sistema operacional, escolha o instalador adequado. No Windows, recomenda-se a opção *User Installer* para a arquitetura do computador, normalmente **x64**.
3. Após o download, execute o arquivo instalador.
4. Avance pelas telas iniciais do assistente de instalação, aceitando os termos de uso quando solicitado.
5. Na tela de tarefas adicionais, é recomendável marcar as opções para adicionar o VS Code ao menu de contexto e ao **PATH**, quando disponíveis. Isso facilita abrir pastas pelo terminal e usar comandos associados ao editor.
6. Conclua a instalação e abra o Visual Studio Code.

Atenção ao nome

Não confunda **Visual Studio Code** com **Visual Studio**. O Visual Studio é uma IDE maior, voltada principalmente para desenvolvimento de software em ambientes Microsoft. Nesta disciplina, será usado o Visual Studio Code.

Depois de abrir o VS Code pela primeira vez, observe a tela inicial e os menus principais. Diferente de outros editores, como o Notepad++, o VS Code trabalha em uma pasta de trabalho, ou seja, os arquivos são organizados dentro de uma pasta que representa o projeto. Para abrir uma pasta, use o menu **File > Open Folder** e selecione a pasta onde os arquivos do projeto serão criados.

Para abrir o terminal integrado, use o menu **Terminal > New Terminal**, ou pressione **Ctrl**

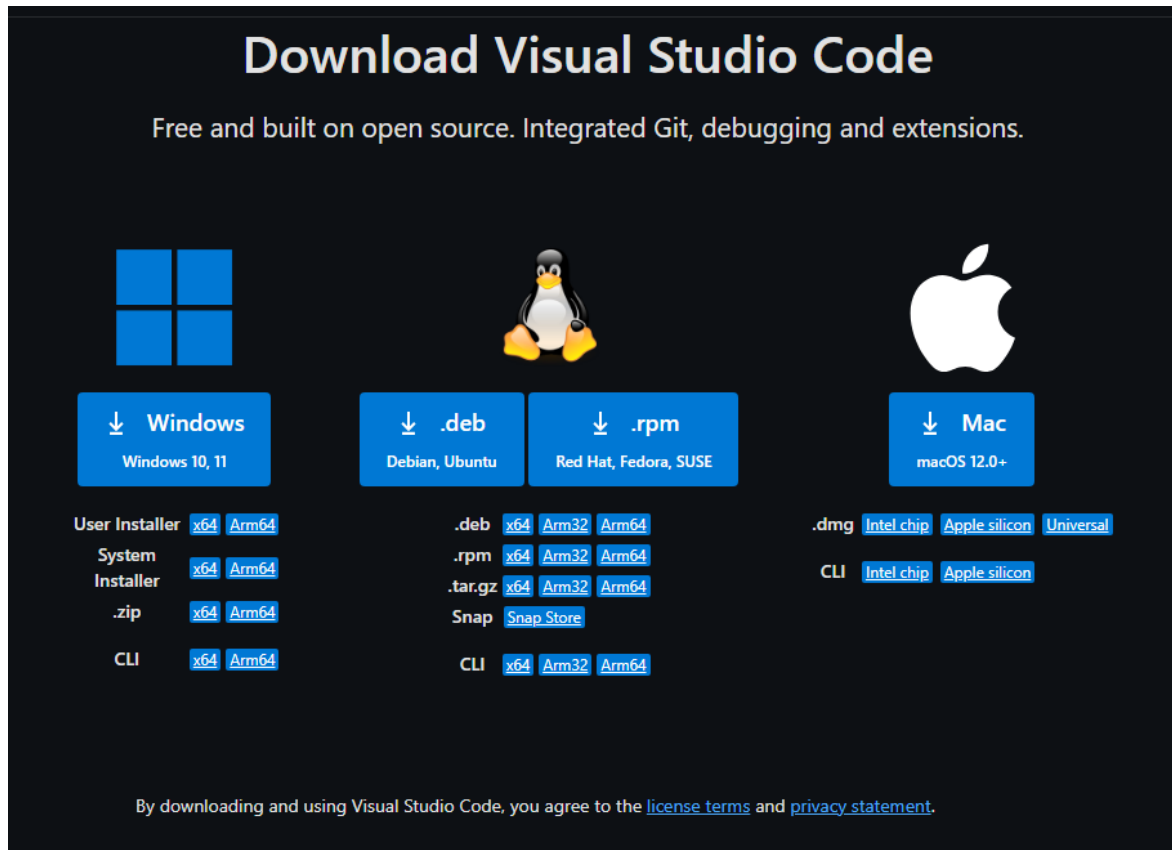


Figura 2.2: Página oficial de download do Visual Studio Code.

+ ‘. Neste momento, não é necessário executar comandos; a abertura do terminal serve apenas para localizar onde os comandos de simulação serão digitados futuramente.

A Figura 2.3 apresenta uma visão geral da interface do VS Code. Para os primeiros exercícios da disciplina, as regiões mais importantes são o explorador de arquivos à esquerda, o editor no centro e o painel inferior, onde o terminal integrado pode ser aberto.

As marcações da figura indicam as principais áreas do editor:

- **Activity Bar:** barra lateral com ícones para alternar entre recursos do VS Code, como explorador de arquivos, busca, controle de versão, execução e extensões.
- **Primary Side Bar:** região lateral que mostra a ferramenta selecionada na *Activity Bar*. No início da disciplina, ela será usada principalmente para visualizar a árvore de arquivos do projeto.
- **Editor Groups:** área central onde os arquivos são abertos e editados. É nessa região que os arquivos VHDL serão escritos.
- **Panel:** painel inferior usado para terminal integrado, mensagens de saída, problemas detectados e outras informações auxiliares. Quando a simulação com GHDL for introduzida, os comandos serão digitados nesse painel.
- **Status Bar:** barra inferior que mostra informações sobre o arquivo aberto e o ambiente, como linguagem reconhecida, codificação, quebras de linha e estado do projeto.

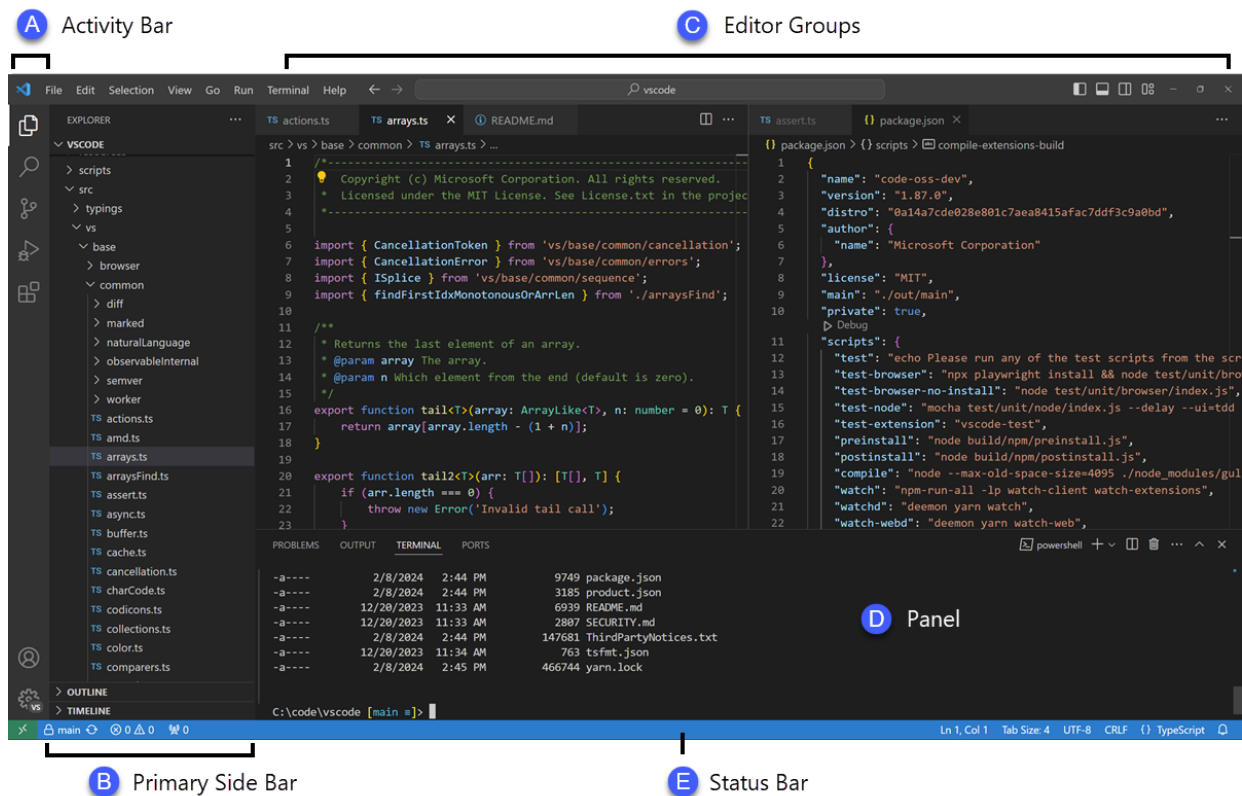


Figura 2.3: Interface do Visual Studio Code com as principais regiões do editor.

2.4 Configuração da extensão VHDL for Professionals

A extensão VHDL for Professionals adiciona recursos de apoio à edição de arquivos VHDL no Visual Studio Code. Ela ajuda com destaque de sintaxe, navegação pelo código e algumas verificações durante a escrita. Isso torna o editor mais confortável, mas é importante separar os papéis: a extensão não substitui o GHDL, não executa simulações e não garante que o circuito esteja correto para síntese.

Para instalar a extensão, abra o Visual Studio Code e siga os passos abaixo:

1. Na *Activity Bar*, clique no ícone de extensões. Esse ícone é mostrado na Figura 2.4. Outra forma de abrir a mesma área é usar o atalho **Ctrl+Shift+X**.
2. Na caixa de busca da área de extensões, digite **VHDL for Professionals**.
3. Localize a extensão chamada **VHDL for Professionals**. O publicador deve ser **ViDE-Software**. A Figura 2.5 mostra a página da extensão no Marketplace do Visual Studio Code, com o nome, o publicador e o botão de instalação.
4. Clique em *Install* e aguarde a conclusão da instalação.
5. Depois da instalação, abra um arquivo com extensão **.vhd** ou **.vhdl**. No canto inferior direito do VS Code, a *Status Bar* deve indicar que a linguagem reconhecida é VHDL.

Se o VS Code não reconhecer automaticamente o arquivo como VHDL, clique no nome da



Figura 2.4: Ícone usado para abrir a área de extensões no *Visual Studio Code*.

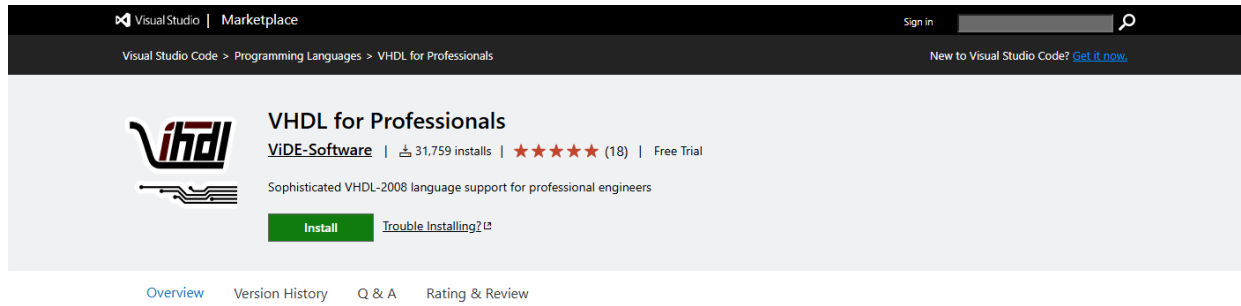


Figura 2.5: Página da extensão *VHDL for Professionals* no Marketplace do *Visual Studio Code*.

linguagem exibido na *Status Bar*, no canto inferior direito, e selecione VHDL manualmente. Essa verificação é simples, mas evita um problema comum: editar um arquivo VHDL como se fosse texto puro, sem destaque de sintaxe e sem os recursos da extensão.

A Figura 2.6 mostra um exemplo de apoio visual oferecido pela extensão durante a edição do código. Esse tipo de indicação ajuda a encontrar problemas enquanto o arquivo ainda está sendo escrito, antes mesmo de executar comandos de simulação.

```
54  label_a: process is
55      constant TheConst: rec := (field1 => 1, field2 => 2); -- colorize initializer
56      variable var: rec := (field1 => 1, field2 => 2);      -- colorize initializer
57      variable p: prot;
58      alias al1 is p;
59      alias al2 is distance;
60  |  alias al3 is constk;
61      alias al4 is sig;
62  begin
63      sig <= (field1 => 1, field2 => 2);
64      sig <= (field1 => 1, field2 => 2) when true else (field1 => 1, field2 => 2);
```

Figura 2.6: Exemplo de destaque e indicação visual em código VHDL no editor.

2.5 Instalação do GHDL

O GHDL é a ferramenta que será usada para analisar, elaborar e simular arquivos VHDL. Diferente da extensão instalada no Visual Studio Code, o GHDL não é apenas um recurso visual do editor: ele executa verificações sobre o código e permite observar o comportamento descrito antes de levar o projeto para uma ferramenta de síntese.

No Windows, a forma mais simples de instalar o GHDL é usando o **winget**, o gerenciador de pacotes do Windows. Antes de instalar, porém, é recomendável pesquisar o pacote disponível no próprio **winget**, pois o identificador do pacote pode mudar com o tempo.

Abra o terminal integrado do Visual Studio Code, ou o PowerShell do Windows, e execute:

Código 2.2: *Pesquisa pelo pacote do GHDL no winget.*

```
1 winget search ghdl
```

Observe a lista exibida e procure uma entrada relacionada ao GHDL. No momento da escrita desta apostila, uma opção comum é o pacote identificado como `ghdl.ghdl.ucrt64.mcode`. Caso esse identificador apareça na busca, a instalação pode ser feita com:

Código 2.3: *Instalação do GHDL pelo winget.*

```
1 winget install --id ghdl.ghdl.ucrt64.mcode --exact
```

Durante a instalação, o `winget` pode solicitar a aceitação dos termos da fonte de pacotes ou do instalador. Leia as mensagens exibidas no terminal e confirme quando apropriado. Ao final da instalação, feche e abra novamente o terminal para garantir que o caminho do GHDL seja reconhecido.

Para verificar se a instalação funcionou, execute:

Código 2.4: *Verificação da instalação do GHDL.*

```
1 ghdl --version
```

Se o comando exibir a versão do GHDL, a instalação está pronta para os próximos exemplos.

Não pule a busca

Mesmo que esta apostila mostre um identificador de pacote, execute primeiro `winget search ghdl`. Essa etapa confirma qual nome está disponível no computador no momento da instalação e evita depender de um identificador que pode ter sido alterado ou substituído no repositório do `winget`.

Se o comando não for reconhecido

Se o Windows informar que `winget` não foi encontrado, verifique se o *App Installer* está instalado e atualizado. Em instalações atuais do Windows 10 e do Windows 11, o `winget` normalmente já está disponível. Se o comando `ghdl -version` não for reconhecido depois da instalação, feche e abra novamente o VS Code ou o PowerShell antes de tentar outra solução.

2.6 Primeira simulação com GHDL

2.7 Instalação do VaporView

2.8 Visualização de formas de onda

2.9 O papel do Quartus no fluxo completo

O Quartus Prime Lite será usado mais adiante na apostila, quando os circuitos simulados passarem a ser implementados fisicamente em uma placa FPGA. Neste capítulo, ele aparece apenas para situar o fluxo completo de desenvolvimento.

Enquanto o GHDL verifica o comportamento descrito em VHDL durante a simulação, o Quartus realiza etapas associadas ao hardware real: síntese lógica, implementação no dispositivo, análise temporal e geração do arquivo que será gravado no FPGA. Essas etapas exigem conhecimentos adicionais, como escolha do dispositivo, atribuição de pinos e interpretação de relatórios da ferramenta.

Por que deixar o Quartus para depois?

Instalar e configurar o Quartus antes de entender simulação pode deslocar o foco do aprendizado. Primeiro, a apostila constrói a base de descrição e verificação em VHDL. Depois, quando o circuito já puder ser validado no computador, o fluxo de implementação em FPGA será introduzido com mais contexto.

2.10 Automação futura com Tcl

2.11 Fluxo de trabalho recomendado

2.12 Problemas comuns e verificação da instalação

2.13 Perguntas para revisão

Referências Bibliográficas

- [1] Peter J. Ashenden. *The Designer's Guide to VHDL*. Morgan Kaufmann, 3 edition, 2008.
- [2] Pong P. Chu. *FPGA Prototyping by VHDL Examples*. Wiley-Interscience, 2008.
- [3] Allen Dewey. VHSIC hardware description language (VHDL) development program. In *Proceedings of the 20th Design Automation Conference*, 1983.
- [4] Doulos. VHDL history and evolution. Technical article.
- [5] IEEE Computer Society. *IEEE Standard VHDL Language Reference Manual*. IEEE, 2009. IEEE Std 1076-2008.
- [6] IEEE Solid-State Circuits Society. The roots of VHDL. *IEEE Solid-State Circuits Magazine*, 2018.
- [7] United States Department of Defense. Requirements for hardware description languages. Technical report, United States Department of Defense, 1981.